

CUDA-Accelerated Program for Real-Time Vocal Isolation

Context & Introduction

This project for CS-4220 aims to use GPU (with CUDA cores) to accelerate computations for a program that separates audio sources in real-time (main objective: vocal isolation) from mixed music samples. The program aims to use U-Net-style neural network or Demucs, short-time fourier transform, neural inference, and inverse STFT running on the accelerated hardware to reduce CPU-GPU memory latency for real-time demonstration.

Audio source separation is gaining traction within the technological industry with recent innovations for hearing aids, surveillance, content productions (music, video, content creations, etc), speech enhancement, and more recently automatic speech recognition (ASR) that helps AI systems identify auditory inputs better.

Some notable commercially available products that utilize the concept include: [Moises.ai](#), Adobe Podcast, and GarageBand. However, a persistent problem that persists as the bottleneck is short-time fourier transformation, as it requires high computation overhead within a short-time frame. A popular CPU implementation named librosa achieved 300-500ms latency, which is not very applicable for real-time performance. Whereas, Nvidia's cuFFT library enabled parallel computation on GPU resulted in recorded ~30x increase in speed. Recent work (Stemuc Audio Forge, BST 2025) confirms that CUDA-accelerated Demucs reduces processing time from ~5 minutes (CPU) to under 10 seconds (GPU), a 30x improvement. This project aims to take into consideration different approaches from researched works and documentations to implement the entire pipeline on accelerated hardware.

Tentative Implementation Plans

Sprint 1: STFT Engine with cuFFT

1. Segment audio stream into overlapping frames: window size 4096 samples, hop size 1024 samples (75% overlap)
2. Write a CUDA kernel for element-wise Hann windowing: each thread multiplies one sample by $w[n] = 0.5 \times (1 - \cos(2\pi n/N))$
3. Execute batched forward FFT using `cufftPlanMany()` and `cufftExecC2C()` on all frames simultaneously in one GPU call
4. Write magnitude/phase extraction kernel: $|X| = \sqrt{re^2 + im^2}$, $\phi = \text{atan2}(im, re)$; use `tanhf` approximation for performance

Sprint 2: Model Preparation & TensorRT Conversion

1. Download Open-Unmix (UMX) pre-trained on MUSDB18 dataset from github.com/sigsep/open-unmix-pytorch
2. Export vocal stem model to ONNX format using `torch.onnx.export()`; convert to TensorRT FP16 engine via `trtexec`
3. Validate: compute Signal-to-Distortion Ratio (SDR) between TensorRT output and PyTorch baseline on MUSDB18 test clips

Sprint 3: Spectral Mask Application & Wiener Filtering

4. TensorRT model outputs soft mask $M(f,t) \in [0,1]$ per frequency bin and time frame
5. Write a CUDA kernel for element-wise complex multiplication: $Y(f,t) = M(f,t) \times X(f,t)$, operating on paired float2 arrays
6. Implement Wiener filter for improved separation quality: $\hat{V}(f,t) = (M^2_v / \sum_s M^2_s) \times X(f,t)$; this further suppresses residual artifacts

Sprint 4: Inverse STFT with Overlap-Add (OLA) Reconstruction

- Execute batched inverse FFT using `cufftExecC2C()` on masked complex STFT frames
- Write overlap-add CUDA kernel: each thread atomically accumulates synthesis window-weighted samples into output buffer using `atomicAdd()` on float array

- Normalize by sum-of-squared windows (constant overlap-add constraint) to recover original amplitude scaling

Sprint 5: CUDA Streams Pipeline for Real-Time Overlapping

- Create 3 CUDA streams: stream_stft (FFT + windowing), stream_nn (TensorRT inference), stream_istft (mask application + iFFT + OLA)
- Chain streams using cudaStreamWaitEvent(): stream_stft signals completion → stream_nn begins; stream_nn signals → stream_istft begins
- Maintain a double-buffered ring buffer of audio chunks to keep all 3 streams continuously occupied

Sprint 6: Benchmarking & Evaluation

- Compare latency per 1-second audio chunk: CPU (librosa STFT + PyTorch) vs. GPU (cuFFT + TensorRT + custom kernels)
- Metrics: latency (ms/chunk), GPU bandwidth utilization, cuFFT time vs. inference time vs. OLA time, SDR on MUSDB18 test set

Available hardware: RTX 3060 in Building 8 Computer Labs, RTX 3080, Fall back (Google Colab (will remove custom kernel & pivot due to a lack of CUDA))

Planned Demonstration Video Plan

A live audio separation demo: a selected song streams through the full GPU pipeline, producing two separate real-time audio outputs: isolated vocals and instrumental. A synchronized spectrogram visualization (matplotlib/PyAudio) shows three panels updating in real-time:

- (1) input mix spectrogram,
- (2) predicted vocal mask
- (3) separated vocal output.

Performance overlay displays: latency per chunk (ms), GPU utilization %, cuFFT stage time vs. inference time.

An optional microphone input allows live singing into the system for real-time separation. A final benchmark table compares GPU vs. CPU latency and SDR score, demonstrating computational differences.

References

- [1] Défossez et al., "Hybrid Spectrogram and Waveform Source Separation" (Demucs v4). github.com/facebookresearch/demucs | SDR 9.00 dB on MUSDB HQ
- [2] Stöter et al., "Open-Unmix: A Reference Implementation for Music Source Separation," JOSS 2019. github.com/sigsep/open-unmix-pytorch
- [3] NVIDIA cuFFT Library Documentation. docs.nvidia.com/cuda/cufft | cuFFT callbacks for windowing: forums.developer.nvidia.com
- [4] Stemuc Audio Forge: AI-Based Music Separation Using Demucs and CUDA Acceleration. BST 2025. researchgate.net/publication/396205016
- [5] Real-Time Low-Latency Music Source Separation Using Hybrid Spectrogram-TasNet. arXiv:2402.17701 (2024). l-acoustics.com/wp-content/uploads/2024/04/real_time_demixer_2024_04_19.pdf